

Documento de Referencia:Esquema Relacional

Proyecto:Aplicación “No Te Cortes”

Sector: Servicios de Estética e Imagen Personal (CNAE 9602)

1. Justificación del Diseño

Para completar la arquitectura de "No Te Cortes", el esquema relacional traduce el diagrama Entidad-Relación a una estructura de tablas normalizada. He aplicado las reglas de transformación para asegurar que no haya redundancia y que la integridad referencial se mantenga, garantizando la consistencia de los datos en los flujos de Cliente, Administrador y Empleados.

2. Definición de Tablas y Atributos

(Notación: PK = Primary Key, FK = Foreign Key)

• TABLA: USUARIOS

- id_user (String / UID): PK, Not Null.
- nombre (String): Not Null.
- apellidos (String): Not Null.
- email (String): Unique, Not Null.
- telefono (String): Not Null.
- direccion (String).
- ciudad (String).
- rol (String): Check (cliente, administrador, empleado).
- fcmToken (String): Para notificaciones Push.
- notificaciones (Boolean): Default True.
- activo (Boolean): Default True.
- createdAt (Long/Timestamp): Fecha de creación del registro.

• TABLA: SERVICIOS

- id_servicio (String): PK, Not Null.
- nombre (String): Unique, Not Null.
- duracion (Int): Not Null, Check (>0).
- precio (Decimal): Not Null, Check (>=0).

• TABLA: PROMOCIONES

- id_promo (String): PK, Not Null.
- descripcion (String): Not Null.
- descuento (Decimal): Not Null, Check (>0).
- dirigidoA (String): Not Null.

- fechaInicio (String/Date): Not Null.
- fechaFin (String/Date): Not Null.
- activa (Boolean): Default False.
- TABLA: CITAS
- id_cita (String): PK, Not Null.
- userId (String): FK (USUARIOS), Not Null.
- id_servicio (String): FK (SERVICIOS), Not Null.
- id_promo (String): FK (PROMOCIONES), Nullable.
- fecha (String/Date): Not Null. - hora (String): Not Null.
- estado (String): Default 'pendiente', Check (pendiente, realizada, no_realizada).
- createdAt (Long/Timestamp): Instante de creación de la reserva.
- TABLA: FECHAS_BLOQUEADAS
- id_bloqueo (String): PK, Not Null.
- fecha (String/Date): Unique, Not Null.
- id_admin (String): FK (USUARIOS), Not Null.

3. Script DDL de Prueba (SQL estándar)

```
-- 1. Tabla de Usuarios
CREATE TABLE USUARIOS (
  id_user VARCHAR(128) PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  telefono VARCHAR(20) NOT NULL,
  direccion VARCHAR(255), ciudad VARCHAR(100),
  rol VARCHAR(20) NOT NULL CHECK (rol IN ('cliente', 'empleado', 'administrador')),
  fcmToken VARCHAR(255),
  notificaciones BOOLEAN DEFAULT TRUE NOT NULL,
  activo BOOLEAN DEFAULT TRUE NOT NULL,
  createdAt BIGINT NOT NULL );

-- 2. Tabla de Servicios
CREATE TABLE SERVICIOS (
  id_servicio VARCHAR(50) PRIMARY KEY,
  nombre VARCHAR(150) UNIQUE NOT NULL,
```

```
duracion INT NOT NULL CHECK (duracion > 0),  
precio DECIMAL(10,2) NOT NULL CHECK (precio >= 0)  
);
```

-- 3. Tabla de Promociones

```
CREATE TABLE PROMOCIONES (  
id_promo VARCHAR(50) PRIMARY KEY,  
descripcion TEXT NOT NULL,  
descuento DECIMAL(10,2) NOT NULL CHECK (descuento > 0),  
dirigidoA VARCHAR(100) NOT NULL,  
fechaInicio VARCHAR(20) NOT NULL,  
fechaFin VARCHAR(20) NOT NULL,  
activa BOOLEAN DEFAULT FALSE NOT NULL );
```

-- 4. Tabla de Citas (Con absorción de FK de promociones)

```
CREATE TABLE CITAS (  
id_cita VARCHAR(50) PRIMARY KEY,  
userId VARCHAR(128) NOT NULL,  
id_servicio VARCHAR(50) NOT NULL,  
id_promo VARCHAR(50),  
fecha VARCHAR(20) NOT NULL,  
hora VARCHAR(10) NOT NULL,  
estado VARCHAR(20) DEFAULT 'pendiente' NOT NULL CHECK (estado IN ('pendiente',  
'realizada', 'no_realizada')),  
createdAt BIGINT NOT NULL,  
FOREIGN KEY (userId) REFERENCES USUARIOS(id_user) ON DELETE RESTRICT,  
FOREIGN KEY (id_servicio) REFERENCES SERVICIOS(id_servicio) ON DELETE  
RESTRICT,  
FOREIGN KEY (id_promo) REFERENCES PROMOCIONES(id_promo) ON DELETE SET  
NULL );
```

-- 5. Tabla de Fechas Bloqueadas

```
CREATE TABLE FECHAS_BLOQUEADAS (  
id_bloqueo VARCHAR(50) PRIMARY KEY,  
fecha VARCHAR(20) UNIQUE NOT NULL,  
motivo VARCHAR(255) NOT NULL,  
id_admin VARCHAR(128) NOT NULL,  
FOREIGN KEY (id_admin) REFERENCES USUARIOS(id_user) ON DELETE  
RESTRICT );
```

4. Reglas de Integridad Referencial

- **Regla de Borrado (RESTRICT):** No se permite la eliminación de registros en USUARIOS o SERVICIOS si tienen citas asociadas. Para gestionar las bajas se utiliza el campo lógico activo.
- **Regla de Borrado en Promociones (SET NULL):** Si una promoción se elimina del catálogo, las citas históricas o pendientes que estuvieran vinculadas a ella no se borran; simplemente su campo id_promo pasa a ser NULL para salvaguardar la integridad de la reserva.

5. Consideraciones para la implementación en Cloud Firestore (NoSQL)

Aunque el diseño lógico se ha formalizado mediante un Esquema Relacional y scripts SQL para garantizar la integridad de las reglas de negocio, la implementación física se llevará a cabo en Cloud Firestore. Para adaptar este modelo relacional a la arquitectura NoSQL, se han definido las siguientes estrategias:

- **Desnormalización y Rendimiento:** Se aplicará desnormalización en la colección citas, duplicando campos como nombreServicio o nombreCliente. Esto evita realizar múltiples consultas (joins manuales) y garantiza una carga instantánea en la interfaz del Administrador y el Cliente.
- **Jerarquía de Subcolecciones:** Para optimizar la seguridad y el acceso, las citas se estructurarán como una subcolección dentro del documento de cada usuario. Esto permite aplicar Reglas de Seguridad de Firebase más granulares.
- **Mapeo de Tipos de Datos:** Los campos definidos como Long (createdAt) y Date se implementarán mediante el tipo Timestamp nativo de Firestore para permitir filtrados y ordenación cronológica eficiente.
- **Identidad unívoca:** El id_user será el UID generado por Firebase Authentication, vinculando la identidad del usuario con su registro en la base de datos de forma unívoca.
- **Integridad en el Lado del Cliente/Funciones:** Al no existir restricciones de clave foránea nativas en NoSQL, las reglas de integridad (como el RESTRICT al borrar servicios) se gestionarán mediante validaciones en la lógica de la aplicación y controladores en Java.